APLICAÇÕES COM COMUNICAÇÃO SEM FIO PARA IoT
(ATIVIDADE DA UNIDADE 2, CAPÍTULO 3)

Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Trilha: Software Embarcado
Mentora: Gabriela Teixeira

1. Enunciados

- **Questão (1):** Com base no código apresentado na aula do capítulo 3, da unidade 2, crie um programa para monitorar os status do botão da placa e enviar, a cada 1 segundo, os status para um servidor. Além disso, como um desafio extra, acrescente algum sensor e envie a informação desse sensor para o servidor.

- **Questão (2):** Com base no código apresentado na aula do capítulo 3, da unidade 2, crie um programa para ler a posição do joystick e enviar a posição x e y para um servidor via Wi-Fi. Além disso, como um desafio extra, crie uma rosa dos ventos imaginaria e envie para o aplicativo a posição (Norte, Sul Leste, Oeste, Nordeste, Sudeste, Noroeste e Sudoeste) selecionada no joystick.

2. Considerações iniciais

Para a solução dos dois problemas propostos foi elaborado um único código, em linguagem C, para estabelecer um protocolo de comunicação TCP/IP com um dispositivo que possua a capacidade de acessar um navegador web. A página deve mostrar os seguintes elementos:

- Estado do botão A da placa BitDogLab (se o botão está ou não pressionado);
- Valor medido pelo sensor de temperatura já embutido na placa (em graus Celsius);
- Valores inteiros correspondentes aos dois potenciômetros do joystick presente na placa BitDogLab (referentes às posições x e y);

- Posição do joystick tendo como referência uma rosa dos ventos, sendo possível identificar oito direções diferentes (Norte, Sul Leste, Oeste, Nordeste, Sudeste, Noroeste e Sudoeste);
- Um desenho da rosa dos ventos para que o usuário da página utilize como referência na hora de mexer o joystick.

Após a definição do que a página deve mostrar ao usuário, foi possível definir quais os requisitos básicos que o programa deve possuir. De forma geral, o programa deve ser capaz de manipular as entradas e saídas (E/S) digitais do sistema, utilizar o conversor analógico-digital (ADC) do microcontrolador e manipular *strings*, tudo isso enquanto gera uma página em HTML que disponibiliza tais informações. A partir do enunciado da Questão (1), definiu-se também como objetivo atualizar o conteúdo da página a cada um segundo.

3. Comentários gerais sobre o código construído

Para a Questão (1), optou-se por associar o estado lógico do botão a uma *string*. Quando o valor lógico do pino ligado ao Botão A é alto, a *string* "não pressionado" aparece na página em HTML desenvolvida (e, de forma análoga, a *string* "pressionado" aparece quando o valor lógico do pino é baixo). Vale ressaltar que isso tem relação com a ligação interna da placa BitDogLab (o pressionar do Botão A liga uma das entradas digitais do circuito direto ao GND da placa). Além disso, um LED verde na placa acende de acordo com o estado do botão (de forma a ajudar na depuração de possíveis erros no código).

Ainda na Questão (1), o valor que chega ao conversor ADC, proveniente do sensor de temperatura, é convertido para graus Celsius e enviado via Wi-Fi junto com as outras informações necessárias. Esse processo é parecido com o que ocorre para os valores das posições x e y da Questão (2), bastando apenas selecionar um canal diferente do ADC para que os dados possam ser captados.

Para que a posição do joystick referente à rosa dos ventos pudesse ser identificada, foi necessário definir, inicialmente, quais os valores referentes às posições x e y aparecem para o joystick parado (lembrando que a leitura do ADC varia de 0 até 4095). Percebeu-se que os valores lidos pelo ADC para o joystick parado ficam ambos em torno de 1760. A partir disso,

definiu-se (empiricamente) a seguinte relação entre os valores de x, y e as posições da rosa dos ventos, descrita pela Tabela 1:

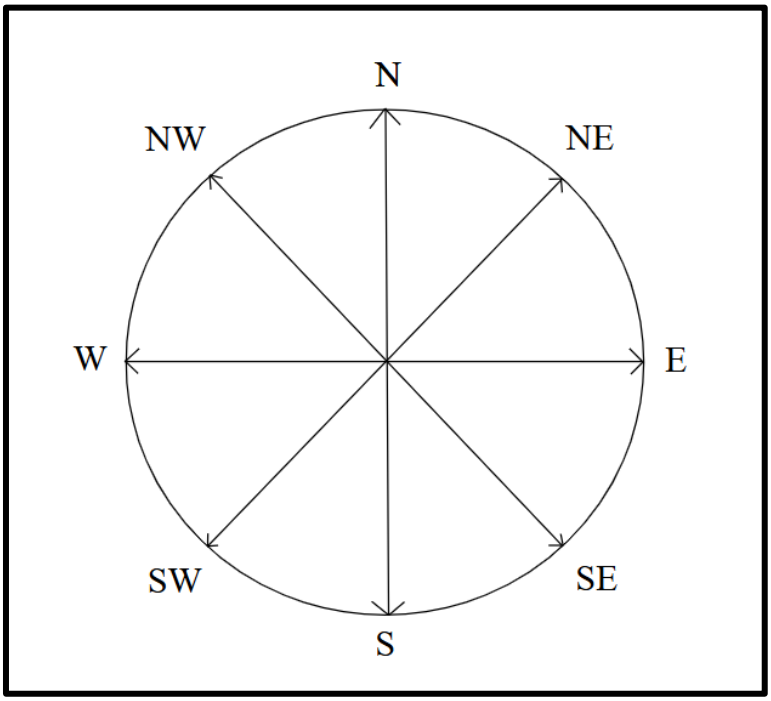
Tabela 1 – Relação entre os valores que representam a posição do joystick e a rosa dos ventos.

	$x < 1260$	$1260 \leq x < 2260$	$x \geq 2260$
$y \geq 2260$	Noroeste (NW)	Norte (N)	Nordeste (NE)
$1260 \leq y < 2260$	Oeste (W)	Centro	Leste (E)
$y < 1260$	Sudoeste (SW)	Sul (S)	Sudeste (SE)

Fonte: Autor (2025).

A imagem de uma rosa dos ventos simplificada foi criada no formato ".SVG", o qual possui fácil integração com HTML. Na Figura 1 é possível ver a imagem desenvolvida para ser mostrada junto com os dados na página HTML que foi criada:

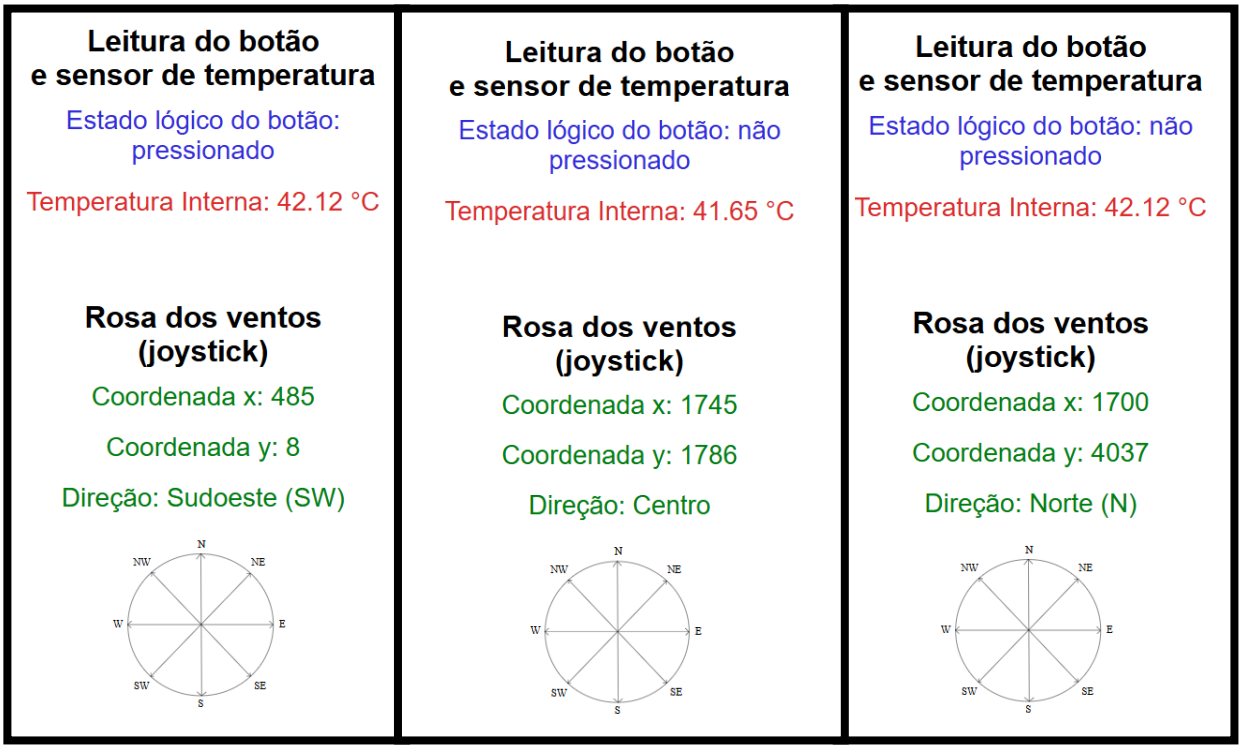
Figura 1 – Desenho da rosa dos ventos criado para a aplicação desenvolvida.



Fonte: Autor (2025).

Por fim, é possível visualizar, na Figura 2, a página em HTML desenvolvida em diversas situações (com o botão pressionado/não pressionado e para algumas das posições da rosa dos ventos):

Figura 2 – Capturas de tela da página HTML desenvolvida para três situações diferentes.



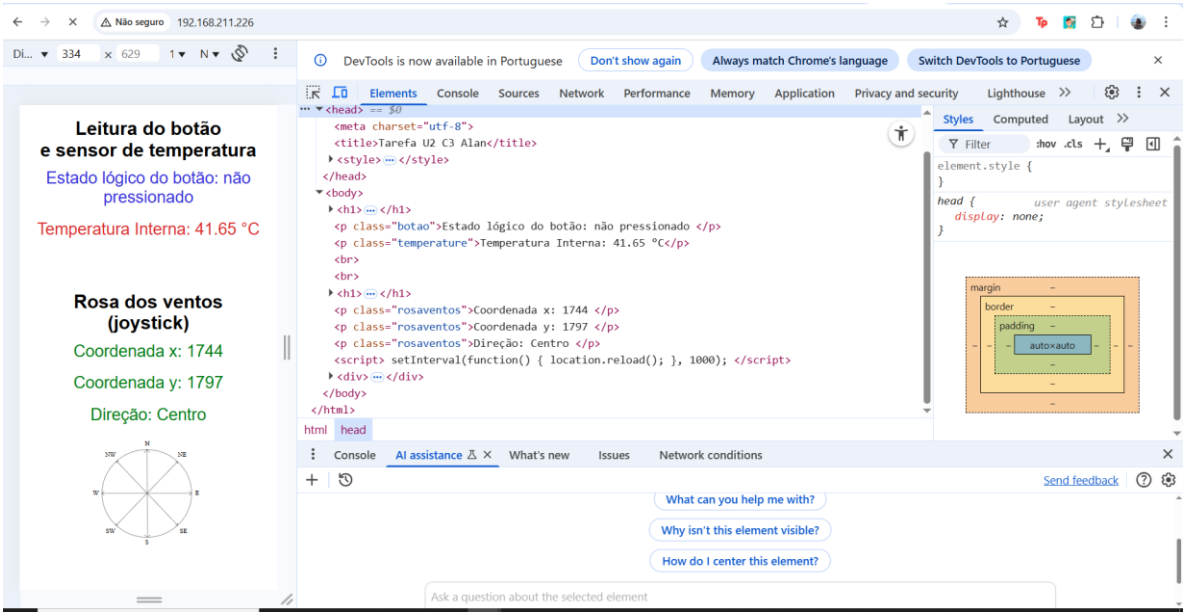
Fonte: Autor (2025).

Diante do exposto, é possível confirmar que o código em C desenvolvido foi capaz de realizar os objetivos propostos pela Questão (1) e pela Questão (2).

4. Comentários específicos sobre protocolos de comunicação e programação web

Ao longo do desenvolvimento das tarefas aqui descritas, foi possível obter um maior contato com conceitos de protocolo de comunicação (em especial o protocolo TCP/IP) e com ferramentas de desenvolvimento web (como HTML e JavaScript). A utilização do ambiente *DevTools* do google chrome, por exemplo, foi essencial para a depuração do aplicação. A Figura 3 ilustra o ambiente em questão sendo utilizado.

Figura 3 – Imagem do ambiente *Chrome DevTools*.



Fonte: Autor (2025).

Para atualizar as informações mostradas na página a cada um segundo, foi utilizada a função "*setInterval(function() { location.reload(); }, 1000)*". Esse comando, em JavaScript, faz com que a página inteira seja recarregada a cada um segundo. Apesar de funcional, procuraram-se outras formas de atualizar os dados da página sem a necessidade de recarregá-la por completo (com outros blocos de código em JavaScript), mas sem sucesso. Suspeita-se que as outras tentativas foram frustradas devido às limitações da biblioteca "lwip.h", a qual é utilizada para gerar uma pilha de protocolo TCP/IP leve. Também não foi descartada a possibilidade de erros no código em JavaScript e no HTML desenvolvidos em outros momentos durante a confecção da atividade.

Também se ressalta a necessidade de alterar o arquivo "lwipopts.h", o qual é utilizado para configurar o comportamento da biblioteca "lwip.h". O tamanho do *buffer* de envio TCP, da fila de pacotes enfileirados para envio e da memória utilizada para a aplicação como um todo foram alterados, assim como algumas outras definições presentes no arquivo. Isso foi importante para que o sistema conseguisse enviar o página HTML de forma adequada, a qual ocupou um número considerável de dados a mais na memória do sistema após a inclusão da figura da rosa dos ventos no formato ".SVG". Para trabalhos futuros, observou-se a necessidade de explorar e entender com mais profundidade as configurações da biblioteca "lwip.h".

Por fim, para comprovar o funcionamento do código, o link a seguir leva para uma pasta no Google Drive que possui os arquivos do projeto desenvolvido, bem como um vídeo do sistema funcionando na placa BitDogLab:

https://drive.google.com/drive/folders/1tyT-vSvUXrU2BA24AcJxftCdIJPrWJCy?usp=sharing

5. Conclusão

A partir do desenvolvimento da atividade, foi possível realizar a integração entre diversas linguagens para o desenvolvimento de aplicações embarcadas e web. Foi possível verificar como gerar, a partir de um código em C, uma página em HTML que é atualizada com uma rotina em JavaScript, demonstrando que, apesar da linguagem C ser elencada como a mais importante dentro do mundo dos sistemas embarcados, o conhecimento de linguagens diversas torna o desenvolvedor mais completo e versátil para o desenvolvimento de produtos mais sofisticados.

Além do exposto, o bom entendimento sobre protocolos de comunicação se torna indispensável dentro de uma realidade onde a Internet das Coisas (IoT) é cada vez mais utilizada. Logo, entender com certo detalhamento como tais protocolos funcionam pode ser o diferencial para se obter uma aplicação leve, funcional e com uma comunicação estável entre os dispositivos envolvidos.

Por fim, vale ressaltar que uma série de dificuldades foi encontrada ao longo do desenvolvimento da atividade, em especial devido à falta de conhecimento sobre ferramentas de programação web. Tal fato deixou clara a necessidade de aprofundamento no tópico, sendo o presente trabalho um bom primeiro contato com esse conjunto de ferramentas.

6. Código em linguagem C desenvolvido para a placa BitDogLab

```c
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 2 Capítulo 3
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/

// Definição das bibliotecas
#include <stdio.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "pico/cyw43_arch.h"

// Biblioteca para geração de uma pilha de protocolos TCP/IP leve
#include "lwip/pbuf.h"
#include "lwip/tcp.h"
#include "lwip/netif.h"

// Configurações da rede wi-fi que será utilizada (ALTERAR AQUI SE MUDAR A
REDE!!!)
#define WIFI_SSID "A54 de Alan"
#define WIFI_PASSWORD "testehoje"


// Definindo os pinos do microcontrolador para o botão e o LED
#define BOTAO 5 // Botão A da placa BitDogLab
#define LED 11 // Pino referente à cor verde do LED RGB da placa BitDogLab
#define LED_PIN CYW43_WL_GPIO_LED_PIN // Pino referente ao LED do módulo Wi-Fi

// Variáveis globais
bool estado_botao = 1; // Variável lógica do botão
uint16_t raw_value; // Variável que irá receber os dados do sensor de
temperatura
const float conversion_factor = 3.3f / (1 << 12); // Fator de conversão para o
valor de temperatura lido
float temperature; // Variável que armazena a temperatura convertida pra graus
Celsius
int joyx, joyy; // Variáveis para armazenar os valores x e y do joystick
char rosa_dos_ventos[20] ="Centro"; // String que armazena a direção da rosa
dos ventos
char estado_botao_string[20] = "não pressionado"; // String que armazena o
estado lógico do botão
```

```
// Função de callback para processar requisições HTTP
static err_t tcp_server_recv(void *arg, struct tcp_pcb *tpcb, struct pbuf *p,
err_t err)
{
    // Verifica se tem algo no buffer para ser processado
    if (!p)
    {
        tcp_close(tpcb);
        tcp_recv(tpcb, NULL);
        return ERR_OK;
    }

    // Alocação de memória
    char *request = (char *)malloc(p->len + 1);

 // Se a alocação de memória falhar, limpa o buffer e retorna um erro
if (!request) {
    pbuf_free(p);
    return ERR_MEM;
};

    // Copia os dados da requisição para a memória recém alocada
    memcpy(request, p->payload, p->len);
    request[p->len] = '\0';


    // Leitura do botão
    estado_botao = gpio_get(BOTAO);
    gpio_put(LED, !estado_botao);

    // Leitura da temperatura interna
    adc_select_input(4);
    raw_value = adc_read();
    temperature = 27.0f - ((raw_value * conversion_factor) - 0.706f) /
0.001721f;

    // Leitura do joystick
    adc_select_input(0);
    joyy = adc_read();
      adc_select_input(1);
    joyx = adc_read();

    // Definindo uma string para descrever o estado do botão
    if(estado_botao==1)
    {
        strcpy(estado_botao_string,"não pressionado");
    }
    else
    {
        strcpy(estado_botao_string,"pressionado");
```

```
    }

    // Definindo a posição na rosa dos ventos
    /*
    OBS: Após uma avaliação empírica, os intervalos [0-1259], [1260-2259] e
[2260-4095] foram selecionados
    para definir a direção da rosa dos ventos a partir da posição do joystick
    */
    if(joyy<1260)
    {
        if(joyx<1260)
        {
            strcpy(rosa_dos_ventos,"Sudoeste (SW)");
        }
        else
        {
            if(joyx>2060)
            {
                strcpy(rosa_dos_ventos,"Sudeste (SE)");
            }
            else
            {
                strcpy(rosa_dos_ventos,"Sul (S)");
            };

        };

    }
    else
    {
        if(joyy>2260)
        {
            if(joyx<1260)
            {
                strcpy(rosa_dos_ventos,"Noroeste (NW)");
            }
            else
            {
                if(joyx>2260)
                {
                    strcpy(rosa_dos_ventos,"Nordeste (NE)");
                }
                else
                {
                    strcpy(rosa_dos_ventos,"Norte (N)");
                };
            };

        }
        else
```

```
        {
            if(joyx<1260)
            {
                strcpy(rosa_dos_ventos,"Oeste (W)");
            }
            else
            {
                if(joyx>2260)
                {
                    strcpy(rosa_dos_ventos,"Leste (E)");
                }
                else
                {
                    strcpy(rosa_dos_ventos,"Centro ");
                };
            };


        };

    };


    // Criando a resposta HTML
    char html[8192];

    snprintf(html, sizeof(html),
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html\r\n"
            "Connection: close\r\n"
            "\r\n"
            "<!DOCTYPE html>\n"
            "<html>\n"
            "<head>\n"
             "<meta charset=\"utf-8\">\n"
            "<title>Tarefa U2 C3 Alan</title>\n"

             // Estrutura geral da página
            "<style>\n"
            "body { font-family: Arial, sans-serif; text-align: center;
margin-top: 50px; }\n"
            "h1 { font-size: 64px; margin-bottom: 30px; }\n"
            "button { font-size: 36px; margin: 10px; padding: 20px 40px;
border-radius: 10px; }\n"
            ".temperature { font-size: 48px; margin-top: 30px; color: #dc2c2c;
}\n"
            ".botao{ font-size: 48px; margin-top: 30px; color: #322cdc; }\n"
            ".rosaventos{ font-size: 48px; margin-top: 30px; color: #008011;
}\n"
            "</style>\n"
```

```
            "</head>\n"
            "<body>\n"
            "<h1>Leitura do botão <br> e sensor de temperatura</h1>\n"
            "<p class=\"botao\">Estado lógico do botão: %s </p>\n"
            "<p class=\"temperature\">Temperatura Interna: %.2f &deg;C</p>\n"
            "<br> <br>\n"
            "<h1>Rosa dos ventos <br> (joystick) </h1>\n"
            "<p class=\"rosaventos\">Coordenada x: %d </p>\n"
            "<p class=\"rosaventos\">Coordenada y: %d </p>\n"
            "<p class=\"rosaventos\">Direção: %s </p>\n"


            // Atualização da página inteira a cada 1 segundo (javascript)
            "<script>\n"
            "setInterval(function() { location.reload(); }, 1000);\n"
            "</script>\n"


             // Desenho da rosa dos ventos
            "<div>\n"
            "<svg width=\"800\" height=\"600\"
xmlns=\"http://www.w3.org/2000/svg\">\n"
            "<g>\n"
            "<title>Layer 1</title>\n"
            "<path stroke=\"#000\" id=\"svg_2\" d=\"m393.90006,381.86675c-
94.1989,0 -170.5,-74.51105 -170.5,-166.5c0,-91.98895 76.30111,-166.5 170.5,-
166.5c94.1989,0 170.5,74.51105 170.5,166.5c0,91.98895 -76.30111,166.5 -
170.5,166.5z\" opacity=\"undefined\" fill=\"none\"/>\n"
            "<line stroke-linecap=\"undefined\" stroke-linejoin=\"undefined\"
id=\"svg_3\" y2=\"381.86675\" x2=\"396.40005\" y1=\"47.86675\"
x1=\"394.40005\" stroke=\"#000\" fill=\"none\"/>\n"
            "<path id=\"svg_4\" d=\"m222.40006,214.86675l341.15388,0\"
opacity=\"undefined\" stroke-linecap=\"undefined\" stroke-
linejoin=\"undefined\" stroke=\"#000\" fill=\"none\"/>\n"
            "<path id=\"svg_5\" d=\"m279.15005,92.61675l232,244\"
opacity=\"undefined\" stroke-linecap=\"undefined\" stroke-
linejoin=\"undefined\" stroke=\"#000\" fill=\"none\"/>\n"
            "<path id=\"svg_6\" d=\"m510.40005,94.86675l-233.49581,241.99162\"
opacity=\"undefined\" stroke-linecap=\"undefined\" stroke-
linejoin=\"undefined\" stroke=\"#000\" fill=\"none\"/>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" stroke-width=\"0\" id=\"svg_7\"
y=\"33.86675\" x=\"387.40005\" stroke=\"#000\" fill=\"#000000\">N</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" stroke-width=\"0\" id=\"svg_8\"
y=\"408.86675\" x=\"387.40005\" stroke=\"#000\" fill=\"#000000\">S</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" stroke-width=\"0\" id=\"svg_9\"
y=\"33.86675\" x=\"387.40005\" stroke=\"#000\" fill=\"#000000\">N</text>\n"
```

```
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_10\" y=\"220.86675\"
x=\"188.40006\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">W</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_11\" y=\"221.86675\"
x=\"578.40005\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">E</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_12\" y=\"76.26676\"
x=\"236.00005\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">NW</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_14\" y=\"75.26676\"
x=\"513.00005\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">NE</text>\n"
            "<text style=\"cursor: move;\" xml:space=\"preserve\" text-
anchor=\"start\" font-family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_17\"
y=\"365.86675\" x=\"519.40005\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">SE</text>\n"
            "<text xml:space=\"preserve\" text-anchor=\"start\" font-
family=\"Noto Sans JP\" font-size=\"24\" id=\"svg_18\" y=\"366.86675\"
x=\"237.40005\" stroke-width=\"0\" stroke=\"#000\"
fill=\"#000000\">SW</text>\n"
            "<line id=\"svg_19\" y2=\"61.26676\" x2=\"384.00005\"
y1=\"48.26676\" x1=\"394.00005\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_20\" y2=\"58.83016\" x2=\"404.04925\"
y1=\"48.27468\" x1=\"394.04933\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_21\" y2=\"373.29136\" x2=\"406.81214\"
y1=\"382.04136\" x1=\"396.49965\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_22\" y2=\"372.97886\" x2=\"385.24965\"
y1=\"382.04136\" x1=\"396.49965\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_23\" y2=\"206.41618\" x2=\"231.8718\"
y1=\"214.72381\" x1=\"222.94879\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_24\" y2=\"222.72375\" x2=\"231.25642\"
y1=\"214.41612\" x1=\"222.6411\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_25\" y2=\"206.41618\" x2=\"555.1925\"
y1=\"215.0315\" x1=\"563.80782\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_26\" y2=\"223.03144\" x2=\"556.11557\"
y1=\"215.33919\" x1=\"563.80782\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_27\" y2=\"101.46657\" x2=\"279.19979\"
y1=\"92.21658\" x1=\"278.44979\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_28\" y2=\"93.96658\" x2=\"286.94978\"
y1=\"91.96658\" x1=\"278.44979\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_29\" y2=\"94.71658\" x2=\"501.99969\"
y1=\"94.46658\" x1=\"510.99968\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line id=\"svg_30\" y2=\"103.46657\" x2=\"510.74968\"
y1=\"94.71658\" x1=\"509.99968\" stroke=\"#000\" fill=\"none\"/>\n"
```

```
            "<line stroke-linecap=\"undefined\" stroke-linejoin=\"undefined\"
id=\"svg_31\" y2=\"328.5165\" x2=\"509.79967\" y1=\"336.0165\"
x1=\"511.29967\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line stroke-linecap=\"undefined\" stroke-linejoin=\"undefined\"
id=\"svg_32\" y2=\"336.0165\" x2=\"501.79967\" y1=\"336.0165\"
x1=\"510.54967\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line stroke-linecap=\"undefined\" stroke-linejoin=\"undefined\"
id=\"svg_33\" y2=\"328.15953\" x2=\"277.32419\" y1=\"336.56265\"
x1=\"277.15613\" stroke=\"#000\" fill=\"none\"/>\n"
            "<line stroke-linecap=\"undefined\" stroke-linejoin=\"undefined\"
id=\"svg_34\" y2=\"336.05847\" x2=\"284.04669\" y1=\"336.56265\"
x1=\"276.98806\" stroke=\"#000\" fill=\"none\"/>\n"
            "</g>\n"
            "</svg>\n"
            "</div>\n"

            "</body>\n"
            "</html>\n",
            estado_botao_string,temperature,joyx,joyy, rosa_dos_ventos);


    // Verificando a página enviada (depuração):
    // printf("HTML gerado:\n%s\n", html);

    // Enviando o HTML
    tcp_write(tpcb, html, strlen(html), TCP_WRITE_FLAG_COPY);
    tcp_output(tpcb);
    free(request);
    pbuf_free(p);
     return ERR_OK;
}

// Função de callback ao aceitar conexões TCP
static err_t tcp_server_accept(void *arg, struct tcp_pcb *newpcb, err_t err)
{
    tcp_recv(newpcb, tcp_server_recv);
    return ERR_OK;
}


// Configurações iniciais do microcontrolador (inicialização do botão, ADC e
LED utilizado para debug)
void setup()
{
        // Inicialização da comunicação padrão do Pi Pico
        stdio_init_all();

        // Inicializando o pino do botão
        gpio_init(BOTAO);
```

```
        gpio_set_dir(BOTAO, GPIO_IN);
        gpio_pull_up(BOTAO);

        // Inicializando o pino do LED
        gpio_init(LED);
        gpio_set_dir(LED, GPIO_OUT);

        // Inicializa o ADC
        adc_init();
        adc_set_temp_sensor_enabled(true);

}

// Função principal do código
int main()
{
    // Configurações iniciais
    setup();

    //Inicializa o chip wi-fi do rapberry pi pico W
     while (cyw43_arch_init())
    {
        printf("Falha ao inicializar Wi-Fi\n");
        sleep_ms(100);
        return -1;
    }

    // Colocando o pi pico W em modo station (vai se conectar ao roteador wi-
fi)
    cyw43_arch_enable_sta_mode();

    // Tenta se conectar À rede wi-fi
    printf("Conectando ao Wi-Fi...\n");
    while (cyw43_arch_wifi_connect_timeout_ms(WIFI_SSID, WIFI_PASSWORD,
CYW43_AUTH_WPA2_AES_PSK, 20000))
    {
        printf("Falha ao conectar ao Wi-Fi\n");
        sleep_ms(100);
        return -1;
    }
    printf("Conectado ao Wi-Fi\n");

    // Mostrando o IP de acesso na rede
    if (netif_default)
    {
        printf("IP do dispositivo: %s\n", ipaddr_ntoa(&netif_default-
>ip_addr));

        // Acende o led do módulo wi-fi se a conexão funcionar
        cyw43_arch_gpio_put(LED_PIN, 1);
```

```
    }

    // Configuração  do servidor TCP
    struct tcp_pcb *server = tcp_new();
    if (!server)
    {
        printf("Falha ao criar servidor TCP\n");
        return -1;
    }

    if (tcp_bind(server, IP_ADDR_ANY, 80) != ERR_OK)
    {
        printf("Falha ao associar servidor TCP à porta 80\n");
        return -1;
    }

    server = tcp_listen(server);
    tcp_accept(server, tcp_server_accept);

    printf("Servidor ouvindo na porta 80\n");

    // Laço infinito para manter e tratar a conexão
    while (true)
    {
        cyw43_arch_poll();
    }

    cyw43_arch_deinit();
    return 0;

}
```

7. Conteúdo do arquivo “lwipopts.h”

```
#ifndef LWIPOPTS_H
#define LWIPOPTS_H

// Configuração mínima para lwIP
#define NO_SYS 1
#define LWIP_SOCKET 0
#define LWIP_NETCONN 0
#define LWIP_TCP 1
#define LWIP_UDP 1
#define MEM_ALIGNMENT 4
#define MEM_SIZE 16384
#define MEMP_NUM_PBUF 16
#define PBUF_POOL_SIZE 32               // Ajuste conforme necessário
#define MEMP_NUM_UDP_PCB 4
#define MEMP_NUM_TCP_PCB 4
#define MEMP_NUM_TCP_SEG 16
#define LWIP_IPV4 1
#define LWIP_ICMP 1
#define LWIP_RAW 1
#define LWIP_DHCP 1
#define LWIP_AUTOIP 1
#define LWIP_DNS 1
#define LWIP_HTTPD 1
#define LWIP_HTTPD_SSI              1  // Habilita SSI
#define LWIP_HTTPD_SUPPORT_POST     1  // Habilita suporte a POST, se
necessário
#define LWIP_HTTPD_DYNAMIC_HEADERS 1
#define HTTPD_USE_CUSTOM_FSDATA 0
#define LWIP_HTTPD_CGI 0           // Desative CGI para economizar memória
#define LWIP_NETIF_HOSTNAME 1


// Aumenta o tamanho do buffer de envio TCP (default: 2048 ou menos)
#define TCP_SND_BUF 16384

// Aumenta a janela TCP (importante para controle de fluxo)
#define TCP_WND 16384

// Número de segmentos (pacotes) que podem estar enfileirados para envio
#define TCP_SND_QUEUELEN 63

// Tamanho máximo de segmento TCP (evita fragmentação)
#define TCP_MSS 1280

#define LWIP_DISABLE_TCP_SANITY_CHECKS 1

#endif /* LWIPOPTS_H */
```

8. Referências

Tanenbaum, A.S.; Feamster, N.; Wetherall, D. **Redes de Computadores**. 6 ed. São Paulo: Pearson, 2021.